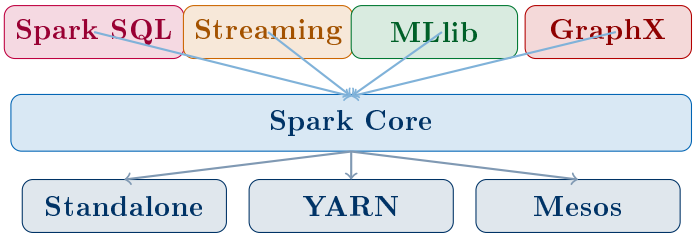
RAPPORT TECHNIQUE

PROJET BIG DATA

TP2 – Traitement par Lot et Streaming avec Apache Spark

Docker · Hadoop YARN · RDD · Spark Batch · Spark Streaming · Java · Maven

Informations du Groupe	
Realise par :	Moussa Sow · Djibril Kane Diallo · Hadja Rougui Sow · Mouhamed El Bachir Diop
Encadreur :	M. Ba
Classe :	Master 1 SRT-GLSI
Module :	Systemes et Architectures Big Data
Ecole :	Ecole Superieure Polytechnique de Dakar
Annee :	2025 / 2026


Spark SQL
Streaming
MLlib
GraphX
Spark Core
Standalone
YARN
Mesos

Contents

Liste des codes source **2**

1 Introduction **3**

2 Mise en Place du Cluster Hadoop avec Docker **4**

2.1 Telechargement de l'Image Docker 4
2.2 Creation du Reseau Bridge 4
2.3 Creation du Conteneur Hadoop Master 5
2.4 Creation des Conteneurs Slaves 5
2.5 Entrer dans le Master et Demarrer Hadoop 6
2.6 Verification que Tout Tourne 7

3 Test de Spark-Shell avec Scala **9**

3.1 Tester avec un Fichier cat 9
3.2 Creer le Repertoire HDFS et Charger le Fichier 9
3.3 Lancer Spark-Shell 10
3.4 Executer le Code Scala – WordCount 11
3.5 Recuperation des Resultats 11
3.6 Fin du Test Spark-Shell — On Passe au Developpement 12

4 Spark Batch en Java — Application WordCount **13**

4.1 Lancement d'IntelliJ IDEA 13
4.2 Creation du Nouveau Projet Maven 13
4.2.1 Cliquer sur New Project 13
4.2.2 Selectionner Maven 14
4.2.3 Verifier le SDK Java 1.8 14
4.2.4 Remplir les Champs Maven 15
4.3 Edition du Fichier pom.xml 15
4.4 Creation du Package tn.insat.tp21 16
4.5 Creation de la Classe Java WordCountTask 17
4.6 Code Java de WordCountTask 17
4.7 Installation de Maven 19
4.8 Installation et Resolution des Dependances Maven 19

4.9 Deploiement sur le Cluster 20
4.9.1 Modifier le Code — Enlever setMaster 20
4.9.2 Packager le JAR 21
4.9.3 Copier le JAR dans le Conteneur Master 22
4.9.4 Lancer le Job 22
4.9.5 Verification des Resultats 23

5 Spark Streaming en Java — Traitement en Temps Reel 24
5.1 Presentation du Spark Streaming 24
5.2 Creer un Nouveau Projet Maven pour le Streaming 24
5.3 Mettre le Code dans pom.xml 25
5.4 Resoudre les Dependances 26
5.5 Creer le Package et la Classe Stream 27
5.6 Code de la Classe Stream.java 27
5.7 Execution et Travail en Temps Reel 29
5.7.1 Lancer Netcat (producteur de flux) 29
5.7.2 Lancer la Classe Stream dans IntelliJ 29
5.7.3 Taper des Donnees dans Netcat 29
5.8 Probleme Rencontre et Solution 31

6 Bilan 32
6.1 Tableau Recapitulatif 33
6.2 Conclusion 34

Listings

2.1 Telechargement de l'image Docker 4
2.2 Creation du reseau bridge Docker 4
2.3 Creation du conteneur hadoop-master 5
2.4 Creation des conteneurs hadoop-slave1 et hadoop-slave2 5
2.5 Connexion au master et demarrage des services Hadoop 6
2.6 Verification des services sur le master et les slaves 7
3.1 Creation du fichier de test avec cat 9
3.2 Creation du repertoire HDFS et chargement de file1.txt 9
3.3 Lancement de spark-shell 10
3.4 Code WordCount en Scala dans spark-shell 11
3.5 Recuperation et affichage des resultats 12
4.1 Contenu du fichier pom.xml du projet Spark Batch 15
4.2 Code complet de WordCountTask.java — Version locale 17
4.3 Installation de Maven sur Ubuntu 19
4.4 Resolution des dependances Maven 19
4.5 WordCountTask.java modifie — Sans setMaster pour le cluster 20
4.6 Generation du fichier JAR avec Maven 21
4.7 Copie du JAR dans hadoop-master 22
4.8 Lancement du job Spark via spark-submit 22
4.9 Verification des resultats du job 23
5.1 pom.xml du projet Spark Streaming 25
5.2 Resolution des dependances Maven pour le Streaming 26
5.3 Code complet de Stream.java — Spark Streaming 27
5.4 Demarrage du flux TCP avec Netcat 29
5.5 Message confirming que le receiver est connecte 29
5.6 Saisie de donnees dans le terminal Netcat 29

Chapter 1

Introduction

Ce rapport presente le travail realise dans le cadre du **TP2 – Traitement par Lot et Streaming avec Spark**, module Big Data du Master 1 SRT-GLSI a l'Ecole Superieure Polytechnique de Dakar, sous la direction de **M. Ba**.

L'objectif est double : maitriser le traitement **par lot (Batch)** avec une application Java WordCount deployee sur un cluster Hadoop/YARN, et le traitement **en temps reel (Streaming)** via Spark Streaming sur un flux TCP simule par Netcat.

Le rapport suit fidelement les etapes effectuees, dans leur ordre chronologique, en les accompagnant des codes utilises, des commandes executees, et des captures d'ecran illustrant chaque resultat obtenu.

Outil	Role	Version
Apache Hadoop	HDFS + YARN	2.7.2
Apache Spark	Moteur de traitement distribue	2.2.1
Docker	Conteneurisation du cluster	17.09.1
IntelliJ IDEA	IDE Java	Ultimate
Java JDK	Langage de developpement	1.8
Maven	Gestion des dependances	3.x
Ubuntu	Systeme hote	–
Netcat (nc)	Simulateur de flux TCP	–

Chapter 2

Mise en Place du Cluster Hadoop avec Docker

2.1 Telechargement de l'Image Docker

La premiere etape consiste a recuperer l'image Docker pre-configuree contenant Hadoop et Spark.

```
docker pull liliasfaxi/spark-hadoop:hv-2.7.2
```

Listing 2.1: Telechargement de l'image Docker

[Capture d'ecran 1]

Telechargement de l'image Docker liliasfaxi/spark-hadoop:hv-2.7.2

Remplacer par la capture reelle

Figure 2.1: Telechargement de l'image Docker liliasfaxi/spark-hadoop:hv-2.7.2

2.2 Creation du Reseau Bridge

On cree un reseau Docker de type **bridge** pour que les conteneurs puissent communiquer entre eux par leurs noms de host.

```
docker network create --driver=bridge hadoop
```

Listing 2.2: Creation du reseau bridge Docker

[Capture d'ecran 2]

Creation du reseau bridge Docker nomme "hadoop"

Remplacer par la capture reelle

Figure 2.2: Creation du reseau bridge Docker nomme "hadoop"

2.3 Creation du Conteneur Hadoop Master

```
docker run -itd \
  --net=hadoop \
  -p 50070:50070 \
  -p 8088:8088 \
  -p 7077:7077 \
  -p 16010:16010 \
  --name hadoop-master \
  --hostname hadoop-master \
  liliasfaxi/spark-hadoop:hv-2.7.2
```

Listing 2.3: Creation du conteneur hadoop-master

[Capture d'ecran 3]

Creation du conteneur hadoop-master avec les ports exposes

Remplacer par la capture reelle

Figure 2.3: Creation du conteneur hadoop-master avec les ports exposes

2.4 Creation des Conteneurs Slaves

```
# Slave 1
docker run -itd \
```

```
  --net=hadoop \
  --name hadoop-slave1 \
  --hostname hadoop-slave1 \
  liliasfaxi/spark-hadoop:hv-2.7.2

# Slave 2
docker run -itd \
  --net=hadoop \
  --name hadoop-slave2 \
  --hostname hadoop-slave2 \
  liliasfaxi/spark-hadoop:hv-2.7.2
```

Listing 2.4: Creation des conteneurs hadoop-slave1 et hadoop-slave2

[Capture d'ecran 4]

Creation des conteneurs hadoop-slave1 et hadoop-slave2

Remplacer par la capture reelle

Figure 2.4: Creation des conteneurs hadoop-slave1 et hadoop-slave2

2.5 Entrer dans le Master et Demarrer Hadoop

```
# Entrer dans le conteneur master
docker exec -it hadoop-master bash

# Lancer les demons HDFS et YARN
./start-hadoop.sh
```

Listing 2.5: Connexion au master et demarrage des services Hadoop

[Capture d'ecran 5]

Entree dans hadoop-master et execution de start-hadoop.sh

Remplacer par la capture reelle

Figure 2.5: Entree dans hadoop-master et execution de start-hadoop.sh

2.6 Verification que Tout Tourne

On utilise `jps` (Java Virtual Machine Process Status) pour verifier que tous les demons sont bien lances.

```
# Sur le master
jps
# Resultat attendu :
# 257  NameNode
# 613  ResourceManager
# 456  SecondaryNameNode

# Sur les slaves (depuis la machine hote)
docker exec -it hadoop-slave1 bash
jps
# Resultat attendu :
# 176  NodeManager
# 65   DataNode
```

Listing 2.6: Verification des services sur le master et les slaves

[Capture d'ecran 6]

Sortie de la commande jps sur le master et sur les slaves

Remplacer par la capture reelle

Figure 2.6: Sortie de la commande jps sur le master et sur les slaves

Resultat

Tous les demons sont lances. Le cluster Hadoop est operationnel.

Chapter 3

Test de Spark-Shell avec Scala

3.1 Tester avec un Fichier cat

On cree un fichier texte simple pour tester Spark.

```
cat > file1.txt << EOF
Hello Spark Wordcount!
Hello Hadoop Also :)
EOF

cat file1.txt
```

Listing 3.1: Creation du fichier de test avec cat

[Capture d'ecran 7]

Creation du fichier file1.txt avec la commande cat

Remplacer par la capture reelle

Figure 3.1: Creation du fichier file1.txt avec la commande cat

3.2 Creer le Repertoire HDFS et Charger le Fichier

```
# Creer le repertoire racine dans HDFS
hadoop fs -mkdir -p .

```

```
# Charger le fichier
hadoop fs -put file1.txt

# Verifier
hadoop fs -ls
```

Listing 3.2: Creation du repertoire HDFS et chargement de file1.txt

[Capture d'ecran 8]

Creation du repertoire HDFS et chargement du fichier file1.txt

Remplacer par la capture reelle

Figure 3.2: Creation du repertoire HDFS et chargement du fichier file1.txt

Attention

Si l'erreur `put: '.': No such file or directory` apparait, il faut d'abord executer `hadoop fs -mkdir -p .` pour creer l'arborescence.

3.3 Lancer Spark-Shell

```
spark-shell
```

Listing 3.3: Lancement de spark-shell

[Capture d'ecran 9]

Lancement de spark-shell — Ecran d'accueil avec la banniere Spark

Remplacer par la capture reelle

Figure 3.3: Lancement de spark-shell — Ecran d'accueil avec la banniere Spark

3.4 Executer le Code Scala – WordCount

On execute le code Scala **ligne par ligne** dans le shell.

```scala
// 1 - Charger le fichier depuis HDFS
val lines = sc.textFile("file1.txt")

// 2 - Separer les mots par les espaces
val words = lines.flatMap(_.split("\\s+"))

// 3 - Associer chaque mot a 1, puis sommer par cle
val wc = words.map(w => (w, 1)).reduceByKey(_ + _)

// 4 - Sauvegarder dans HDFS
wc.saveAsTextFile("file1.count")
```

Listing 3.4: Code WordCount en Scala dans spark-shell

[Capture d'ecran 10]

Execution du code Scala WordCount ligne par ligne dans spark-shell

Remplacer par la capture reelle

Figure 3.4: Execution du code Scala WordCount ligne par ligne dans spark-shell

Ce pipeline realise :

1. **textFile** : cree un RDD ou chaque element est une ligne du fichier.
2. **flatMap + split** : eclate chaque ligne en mots individuels.
3. **map** : produit des paires `(mot, 1)`.
4. **reduceByKey** : additionne les 1 pour chaque mot identique.
5. **saveAsTextFile** : action qui declenche le calcul et sauvegarde.

3.5 Recuperation des Resultats

```
# Sortir de spark-shell avec Ctrl-C
# Telecharger le resultat depuis HDFS
hadoop fs -get file1.count

# Lire les resultats
cat file1.count/part-00000
cat file1.count/part-00001
```

Listing 3.5: Recuperation et affichage des resultats

[Capture d'ecran 11]

Recuperation du repertoire file1.count et affichage des resultats

Remplacer par la capture reelle

Figure 3.5: Recuperation du repertoire file1.count et affichage des resultats

Resultat

```
(Hello,2)
(Wordcount!,1)
(Spark,1)
(:),1)
(Also,1)
(Hadoop,1)
```

"Hello" apparait 2 fois car present sur les deux lignes. Test Spark-Shell reussi !

3.6 Fin du Test Spark-Shell — On Passe au Developpement

Maintenant qu'on a fini avec le test de Spark-Shell, on passe au developpement d'une vraie application Java avec IntelliJ IDEA.

Chapter 4

Spark Batch en Java — Application Word-Count

4.1 Lancement d'IntelliJ IDEA

On ouvre IntelliJ IDEA pour creer notre projet Spark Batch.

[Capture d'ecran 12]

Lancement d'IntelliJ IDEA

Remplacer par la capture reelle

Figure 4.1: Lancement d'IntelliJ IDEA

4.2 Creation du Nouveau Projet Maven

4.2.1 Cliquer sur New Project

Dans IntelliJ, aller dans **File** → **New** → **Project**.

[Capture d'ecran 13]

Clic sur New Project dans IntelliJ IDEA

Remplacer par la capture reelle

Figure 4.2: Clic sur New Project dans IntelliJ IDEA

4.2.2 Selectionner Maven

Dans le menu a gauche, selectionner **Maven**.

[Capture d'ecran 14]

Selection de Maven dans le menu gauche d'IntelliJ

Remplacer par la capture reelle

Figure 4.3: Selection de Maven dans le menu gauche d'IntelliJ

4.2.3 Verifier le SDK Java 1.8

S'assurer que le **Project SDK** est bien configure sur **Java 1.8**.

Attention

Si Java 1.8 n'est pas selectionne, il faut cliquer sur le menu deroulant "Project SDK" et choisir la version 1.8, ou la telecharger si absente.

[Capture d'ecran 15]

Verification que le Project SDK est sur Java 1.8

Remplacer par la capture reelle

Figure 4.4: Verification que le Project SDK est sur Java 1.8

4.2.4 Remplir les Champs Maven

Apres avoir clique sur **Next**, remplir les champs :

Champ	Valeur
GroupId	`spark.batch`
ArtifactId	`wordcount`
Version	`1`

[Capture d'ecran 16]

Remplissage des champs Maven : GroupId=spark.batch, ArtifactId=wordcount, Version=1

Remplacer par la capture reelle

Figure 4.5: Remplissage des champs Maven : GroupId=spark.batch, ArtifactId=wordcount, Version=1

4.3 Edition du Fichier pom.xml

On edite le fichier `pom.xml` avec les dependances Spark :

```
<project>
  <groupId>spark.batch</groupId>
```

```
  <artifactId>wordcount</artifactId>
  <version>1</version>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-core_2.11</artifactId>
      <version>2.1.0</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-log4j12</artifactId>
      <version>1.7.22</version>
    </dependency>
  </dependencies>
</project>
```

Listing 4.1: Contenu du fichier pom.xml du projet Spark Batch

[Capture d'ecran 17]

Edition du fichier pom.xml dans IntelliJ avec les dependances Spark

Remplacer par la capture reelle

Figure 4.6: Edition du fichier pom.xml dans IntelliJ avec les dependances Spark

4.4 Creation du Package tn.insat.tp21

Sous `src/main/java`, creer le package **tn.insat.tp21**. Ce package est necessaire pour contenir notre classe Java.

[Capture d'ecran 18]

Creation du nouveau package tn.insat.tp21 dans IntelliJ

Remplacer par la capture reelle

Figure 4.7: Creation du nouveau package tn.insat.tp21 dans IntelliJ

4.5 Creation de la Classe Java WordCountTask

Dans le package `tn.insat.tp21`, creer la classe **WordCountTask**.

[Capture d'ecran 19]

Creation de la classe Java WordCount-Task dans le package tn.insat.tp21

Remplacer par la capture reelle

Figure 4.8: Creation de la classe Java WordCountTask dans le package tn.insat.tp21

4.6 Code Java de WordCountTask

On insere le code suivant dans la classe `WordCountTask` :

```
package tn.insat.tp21;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import scala.Tuple2;
import java.util.Arrays;
```

```java
import static jersey.repackaged.com.google.common.base.Preconditions
    .checkArgument;

public class WordCountTask {

  private static final Logger LOGGER =
      LoggerFactory.getLogger(WordCountTask.class);

  public static void main(String[] args) {
    checkArgument(args.length > 1,
        "Please provide the path of input file and output dir."
            );
    new WordCountTask().run(args[0], args[1]);
  }

  public void run(String inputFilePath, String outputDir) {

    // Mode local pour le test en local
    String master = "local[*]";
    SparkConf conf = new SparkConf()
        .setAppName(WordCountTask.class.getName())
        .setMaster(master);

    JavaSparkContext sc = new JavaSparkContext(conf);

    // Lire le fichier d'entree
    JavaRDD<String> textFile = sc.textFile(inputFilePath);

    // Pipeline : split -> map -> reduce
    JavaPairRDD<String, Integer> counts = textFile
        .flatMap(s -> Arrays.asList(s.split(" ")).iterator())
        .mapToPair(word -> new Tuple2<>(word, 1))
        .reduceByKey((a, b) -> a + b);

    // Sauvegarder le resultat
    counts.saveAsTextFile(outputDir);
  }
}
```

Listing 4.2: Code complet de WordCountTask.java — Version locale

[Capture d'ecran 20]

Code Java WordCountTask insere dans IntelliJ IDEA

Remplacer par la capture reelle

Figure 4.9: Code Java WordCountTask insere dans IntelliJ IDEA

4.7 Installation de Maven

On installe Maven sur la machine Ubuntu.

```
sudo apt update
sudo apt install maven -y
mvn --version
```

Listing 4.3: Installation de Maven sur Ubuntu

[Capture d'ecran 21]

Installation de Maven sur la machine Ubuntu

Remplacer par la capture reelle

Figure 4.10: Installation de Maven sur la machine Ubuntu

4.8 Installation et Resolution des Dependances Maven

```
mvn dependency:resolve
```

Listing 4.4: Resolution des dependances Maven

[Capture d'ecran 22]

Telechargement et installation des dependances Maven (spark-core, slf4j)

Remplacer par la capture reelle

Figure 4.11: Telechargement et installation des dependances Maven (spark-core, slf4j)

[Capture d'ecran 23]

Resolution complete des dependances Maven — BUILD SUCCESS

Remplacer par la capture reelle

Figure 4.12: Resolution complete des dependances Maven — BUILD SUCCESS

Resultat

Les dependances sont resolues avec succes. Le projet compile. Spark Batch en mode local est **TERMINE** !

4.9 Deploiement sur le Cluster

4.9.1 Modifier le Code — Enlever setMaster

Pour lancer le job sur le cluster, on modifie la methode `run` dans `WordCountTask.java` en **enlevant la ligne setMaster** :

```
public void run(String inputFilePath, String outputDir) {

  // Plus de .setMaster() ici : fourni par spark-submit
  SparkConf conf = new SparkConf()
      .setAppName(WordCountTask.class.getName());

  JavaSparkContext sc = new JavaSparkContext(conf);

```

```
    JavaRDD<String> textFile = sc.textFile(inputFilePath);

    JavaPairRDD<String, Integer> counts = textFile
        .flatMap(s -> Arrays.asList(s.split("\t")).iterator())
        .mapToPair(word -> new Tuple2<>(word, 1))
        .reduceByKey((a, b) -> a + b);

    counts.saveAsTextFile(outputDir);
}
```

Listing 4.5: WordCountTask.java modifie — Sans setMaster pour le cluster

[Capture d'ecran 24]

Modification de WordCountTask.java — Suppression de la ligne setMaster

Remplacer par la capture reelle

Figure 4.13: Modification de WordCountTask.java — Suppression de la ligne setMaster

4.9.2 Packager le JAR

```
mvn package install
```

Listing 4.6: Generation du fichier JAR avec Maven

[Capture d'ecran 25]

Maven package install — Generation du wordcount-1.jar dans le repertoire target

Remplacer par la capture reelle

Figure 4.14: Maven package install — Generation du wordcount-1.jar dans le repertoire target

4.9.3 Copier le JAR dans le Conteneur Master

```
docker cp target/wordcount-1.jar \
    hadoop-master:/root/wordcount-1.jar
```

Listing 4.7: Copie du JAR dans hadoop-master

[Capture d'ecran 26]

Copie du fichier wordcount-1.jar dans le conteneur hadoop-master

Remplacer par la capture reelle

Figure 4.15: Copie du fichier wordcount-1.jar dans le conteneur hadoop-master

4.9.4 Lancer le Job

```
spark-submit \
  --class tn.insat.tp21.WordCountTask \
  --master local \
  --driver-memory 4g \
  --executor-memory 2g \
  --executor-cores 1 \
  wordcount-1.jar \
  input/purchases.txt \
  output
```

Listing 4.8: Lancement du job Spark via spark-submit

[Capture d'ecran 27]

Lancement du job Spark Batch via spark-submit — Logs d'execution

Remplacer par la capture reelle

Figure 4.16: Lancement du job Spark Batch via spark-submit — Logs d'execution

4.9.5 Verification des Resultats

Le job a bien tourne. On verifie les resultats :

```
hadoop fs -get output3
cat output3/part-00000 | head -20
```

Listing 4.9: Verification des resultats du job

[Capture d'ecran 28]

Commande hadoop fs -get output3
et cat output3/part-00000 | head -20

Remplacer par la capture reelle

Figure 4.17: Commande hadoop fs -get output3 et cat output3/part-00000 | head -20

Resultat

```
(166.92,82)
(116.84,86)
(379.92,78)
(411.79,95)
(487.01,72)
...
```

Spark Batch en Java : TERMINE avec succes !

Chapter 5

Spark Streaming en Java — Traitement en Temps Reel

5.1 Presentation du Spark Streaming

Spark Streaming permet le traitement de donnees en quasi temps reel. Il decoupe le flux entrant en **micro-batches** d'une duree fixe (ici 1 seconde), traites chacun comme un RDD classique par le moteur Spark.

5.2 Creer un Nouveau Projet Maven pour le Streaming

Dans IntelliJ, on ouvre un nouveau projet : **File** → **New** → **Project**

On configure :

Champ	Valeur
Build system	Maven
JDK	Java 1.8
GroupId	`spark.streaming`
ArtifactId	`stream`
Version	`1`

[Capture d'ecran 29]

Creation du nouveau projet Maven spark.streaming dans IntelliJ IDEA

Remplacer par la capture reelle

Figure 5.1: Creation du nouveau projet Maven spark.streaming dans IntelliJ IDEA

5.3 Mettre le Code dans pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
␣␣␣␣http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>spark.streaming</groupId>
  <artifactId>stream</artifactId>
  <version>1</version>

  <dependencies>
    <dependency>
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-core_2.11</artifactId>
      <version>2.2.1</version>
    </dependency>
    <dependency>
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-streaming_2.11</artifactId>
      <version>2.2.1</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
```

```
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.1</version>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Listing 5.1: pom.xml du projet Spark Streaming

[Capture d'ecran 30]

Edition du pom.xml du projet Spark Streaming dans IntelliJ

Remplacer par la capture reelle

Figure 5.2: Edition du pom.xml du projet Spark Streaming dans IntelliJ

5.4 Resoudre les Dependances

```
mvn dependency:resolve
```

Listing 5.2: Resolution des dependances Maven pour le Streaming

[Capture d'ecran 31]

Resolution des dependances Maven pour spark-streaming — BUILD SUCCESS

Remplacer par la capture reelle

Figure 5.3: Resolution des dependances Maven pour spark-streaming — BUILD SUCCESS

5.5 Creer le Package et la Classe Stream

Dans IntelliJ, sous `src/main/java` :

- Creer le package : **tn.insat.tp22**
- Creer la classe : **Stream**

[Capture d'ecran 32]

Creation du package tn.insat.tp22
et de la classe Stream dans IntelliJ

Remplacer par la capture reelle

Figure 5.4: Creation du package tn.insat.tp22 et de la classe Stream dans IntelliJ

5.6 Code de la Classe Stream.java

```java
package tn.insat.tp22;

import org.apache.spark.SparkConf;
import org.apache.spark.streaming.Durations;
import org.apache.spark.streaming.api.java.JavaDStream;
import org.apache.spark.streaming.api.java.JavaPairDStream;
import org.apache.spark.streaming.api.java.
    JavaReceiverInputDStream;
import org.apache.spark.streaming.api.java.JavaStreamingContext
    ;
import scala.Tuple2;
import java.util.Arrays;

public class Stream {

  public static void main(String[] args)
      throws InterruptedException {

    SparkConf conf = new SparkConf()
        .setAppName("NetworkWordCount")
```

```
        .setMaster("local[*]");

    // Contexte Streaming avec un micro-batch toutes les
       secondes
    JavaStreamingContext jssc =
        new JavaStreamingContext(conf, Durations.seconds(1));

    // Ecouter sur le socket TCP localhost:9999
    JavaReceiverInputDStream<String> lines =
        jssc.socketTextStream("localhost", 9999);

    // Decouper les lignes en mots
    JavaDStream<String> words =
        lines.flatMap(
            x -> Arrays.asList(x.split(" ")).iterator());

    // Creer les paires (mot, 1)
    JavaPairDStream<String, Integer> pairs =
        words.mapToPair(s -> new Tuple2<>(s, 1));

    // Compter les occurrences par mot sur chaque micro-batch
    JavaPairDStream<String, Integer> wordCounts =
        pairs.reduceByKey((i1, i2) -> i1 + i2);

    // Afficher les resultats en temps reel
    wordCounts.print();

    jssc.start();
    jssc.awaitTermination();
  }
}
```

Listing 5.3: Code complet de Stream.java — Spark Streaming

[Capture d'ecran 33]

Code Java complet de Stream.java dans IntelliJ IDEA

Remplacer par la capture reelle

Figure 5.5: Code Java complet de Stream.java dans IntelliJ IDEA

5.7 Execution et Travail en Temps Reel

5.7.1 Lancer Netcat (producteur de flux)

Dans un **premier terminal**, lancer Netcat pour simuler un flux :

```
# Version recommandee : boucle pour garder nc actif
while true; do nc -l 9999; done
```

Listing 5.4: Demarrage du flux TCP avec Netcat

5.7.2 Lancer la Classe Stream dans IntelliJ

Executer la classe `Stream` dans IntelliJ et attendre ce message :

```
INFO SocketReceiver: Connecting to localhost:9999
INFO SocketInputDStream: Connected to localhost:9999
```

Listing 5.5: Message confirming que le receiver est connecte

5.7.3 Taper des Donnees dans Netcat

```
Bonjour Monsieur Ba nous sommes le groupe 2
Moussa Sow Djibril Kane Diallo Hadja Rougui Sow El Bachir Diop
```

Listing 5.6: Saisie de donnees dans le terminal Netcat

[Capture d'ecran 34]

Saisie dans Netcat et affichage du comptage en temps reel dans IntelliJ

Remplacer par la capture reelle

Figure 5.6: Saisie dans Netcat et affichage du comptage en temps reel dans IntelliJ

Resultat

```
-------------------------------------------
Time: 1772325060000 ms
-------------------------------------------
(Bonjour,1)
(Monsieur,1)
(Ba,1)
(nous,1)
(sommes,1)
(le,1)
(groupe,1)
(2,1)
```

Le travail en temps reel fonctionne ! Spark comptabilise les mots de chaque micro-batch et affiche le resultat chaque seconde.

[Capture d'ecran 35]

Resultats complets du Spark Streaming — Comptage temps reel confirme

Remplacer par la capture reelle

Figure 5.7: Resultats complets du Spark Streaming — Comptage temps reel confirme

5.8 Probleme Rencontre et Solution

Probleme rencontre

Lors du premier test, aucune donnee ne s'affichait dans la console Spark meme apres avoir tape des phrases dans Netcat. Les logs indiquaient :

```
ShuffleBlockFetcherIterator: Getting 0 non-empty blocks
SocketReceiver: Closed socket to localhost:9999
java.net.SocketException: Socket closed
```

La commande `nc -lk` se fermait prematurement sur Ubuntu avant que Spark puisse traiter les donnees.

Solution apportee

Remplacer `nc -lk 9999` par une boucle de relance automatique :

```
while true; do nc -l 9999; done
```

Et respecter l'ordre : lancer Netcat **avant** Stream.java, puis attendre le message de connexion avant d'envoyer des donnees.

Chapter 6

Bilan

6.1 Tableau Recapitulatif

Etape	Description	Statut
Telechargement image	Docker pull Hadoop/Spark	**OK**
Reseau bridge	docker network create hadoop	**OK**
Conteneurs	Master + 2 slaves crees	**OK**
Demarrage Hadoop	start-hadoop.sh + jps verifie	**OK**
Test cat + HDFS	Fichier cree et charge	**OK**
Spark-Shell Scala	WordCount interactif	**OK**
Recuperation resultats	hadoop fs -get + cat	**OK**
IntelliJ + Maven Batch	Projet cree, pom.xml configure	**OK**
Package tn.insat.tp21	Cree dans IntelliJ	**OK**
WordCountTask.java	Code Java ecrit et teste	**OK**
Maven install	Dependances resolues	**OK**
Spark Batch local	Fonctionne en local	**OK**
Modification cluster	setMaster supprime	**OK**
Copie JAR + spark-submit	Job tourne sur le cluster	**OK**
Verification output3	head -20 confirme le resultat	**OK**
Nouveau projet Stream	Maven spark.streaming cree	**OK**
pom.xml Streaming	Dependance spark-streaming ajoutee	**OK**
Package tn.insat.tp22	Cree dans IntelliJ	**OK**
Stream.java	Code ecrit et execute	**OK**
Streaming temps reel	Fonctionne avec Netcat	**OK**

6.2 Conclusion

Ce TP nous a permis de maitriser les deux paradigmes fondamentaux du Big Data avec **Apache Spark** :

- Le **traitement par lot** via une application Java WordCount deployee sur un cluster Hadoop/YARN via Docker.
- Le **traitement en temps reel** via Spark Streaming, en ecoutant un flux TCP et en affichant les resultats seconde par seconde.

Les difficultes rencontrees — notamment le comportement de Netcat sur Ubuntu — ont ete resolues de maniere autonome, ce qui temoigne de la capacite du groupe a diagnostiquer et corriger des problemes en environnement Big Data.

TP2 realise integralement avec succes

Cluster Docker | Spark-Shell | Spark Batch Java | Spark Streaming Java